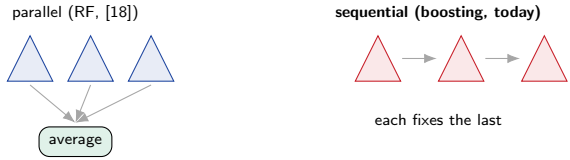
From averaging to fixing mistakes

The random forest ([18]) grew trees **in parallel** and averaged them to cut variance. **What if each new tree instead fixes what the previous ones got wrong?**



That sequential "fix the leftover error" idea is **boosting** – the method that ruled tabular ML for a decade.

Outline

The boosting idea

Combine many weak learners, each fixing what the others miss.

The boosting question

> *Can many* **weak** *learners – each barely better than a coin flip – be combined into one* **strong** *learner?*
>
> (Kearns & Schapire, 1989.)

Boosting's answer: yes – if each learner focuses on the examples the previous ones got wrong, their combination can be arbitrarily accurate.

The base learner is a **shallow tree** (often a stump) – the "atom" from [17]. We add many of them, one at a time.

Bagging vs boosting

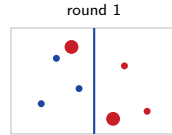
	Bagging / RF ([18])	**Boosting (today)**
trees built	in **parallel**, independent	in **sequence**, dependent
each tree sees	a bootstrap resample	the previous ones' **errors**
combine by	averaging / voting	adding (a weighted sum)
mainly cuts	variance	bias
more trees	safe (plateaus)	**can overfit**

Mirror of [18]. RF bags **deep** (high-variance) trees and averages the variance away. Boosting boosts **shallow** (high-bias) trees – each only nudges the fit, and the bias is removed *across rounds*. Opposite ingredient, opposite target.

AdaBoost, in one picture

The original boosting algorithm (1995):

1. Fit a stump on equally-weighted points.
2. **Up-weight** the points it got wrong, down-weight the rest.
3. Refit on the new weights; repeat.
4. Final prediction = **vote weighted** by each stump's accuracy.



Misclassified points grow; next stump must attend to them.

AdaBoost = boosting with the exponential loss. Gradient boosting (next) generalizes it to *any* loss – both are the same "forward stagewise additive model" (AdaBoost just predates the gradient view). The stump-weight arithmetic $\alpha = \frac{1}{2} \ln \frac{1-\text{err}}{\text{err}}$ is in the HW.

Gradient boosting

Fit the leftover error, add a slice, repeat – and watch it work.

The idea: fit the residuals

Predict apartment rent from area. The recipe:

1. Start simple: $F_0 = \text{mean}(y)$.
2. Compute **residuals** $r = y - F_0$ – what is still wrong.
3. Fit a **small tree** to the residuals.
4. Add it, shrunk by a **learning rate** η: $F_1 = F_0 + \eta\,(\text{tree})$.
5. Recompute residuals; repeat.

Each tree learns the *previous* ensemble's mistakes. The fit creeps toward the data one correction at a time.

By hand: round 1 (Yerevan rent vs area)

Init: $F_0 = \text{mean}(y) = \mathbf{304}$ kAMD (a flat line).

Residuals: $r_i = y_i - 304$.

First stump asks "area ≤ 100?" and predicts each side's *residual mean*:

$$\text{left} = -17, \qquad \text{right} = +78.$$

Update with $\eta = 0.4$:

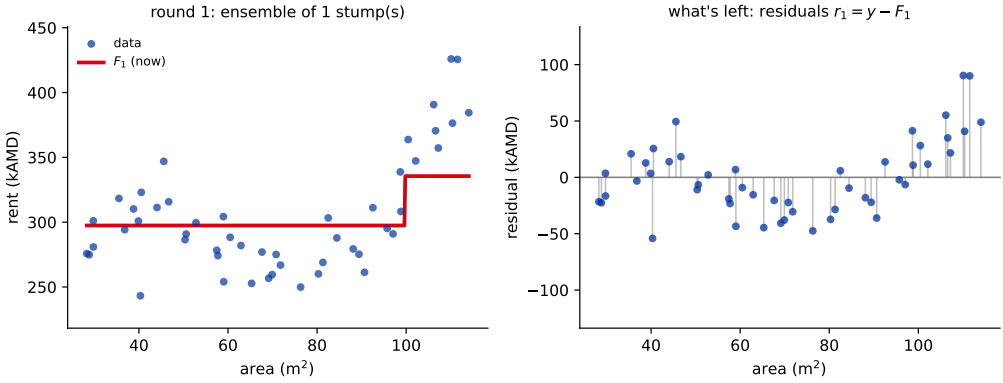
$$F_1 = 304 + 0.4 \times \begin{cases} -17 & \text{area} \leq 100 \\ +78 & \text{area} > 100 \end{cases}$$

$$\Rightarrow \; F_1 = \begin{cases} \mathbf{297} \\ \mathbf{336} \end{cases}$$

One stump nudges the flat line into **two steps**. The big positive residuals at large area shrink first – the model fixes its worst errors first.

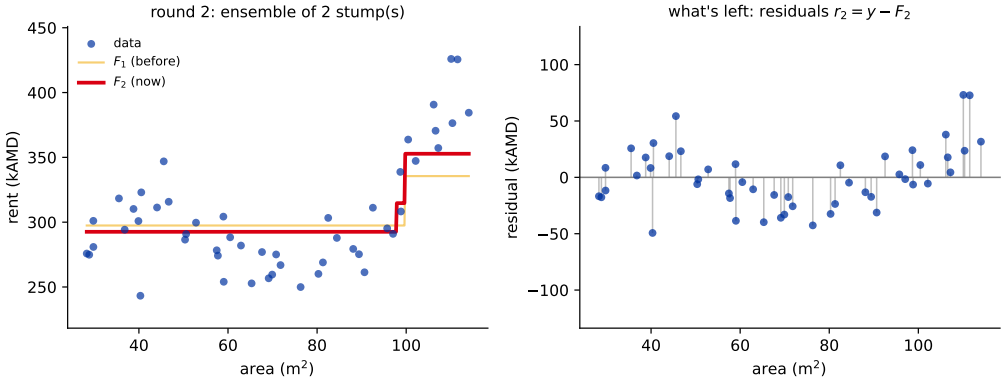
These are the real numbers behind round 1 of the next slide.

Watch it fit: gradient boosting, round by round



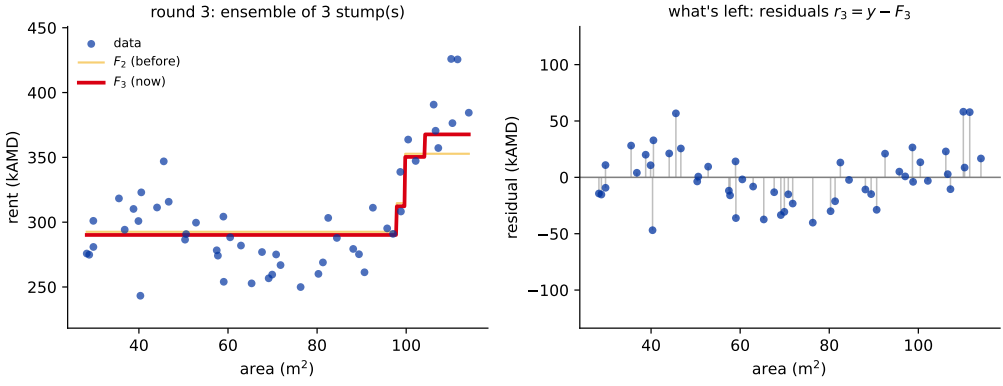
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6.)*

Watch it fit: gradient boosting, round by round



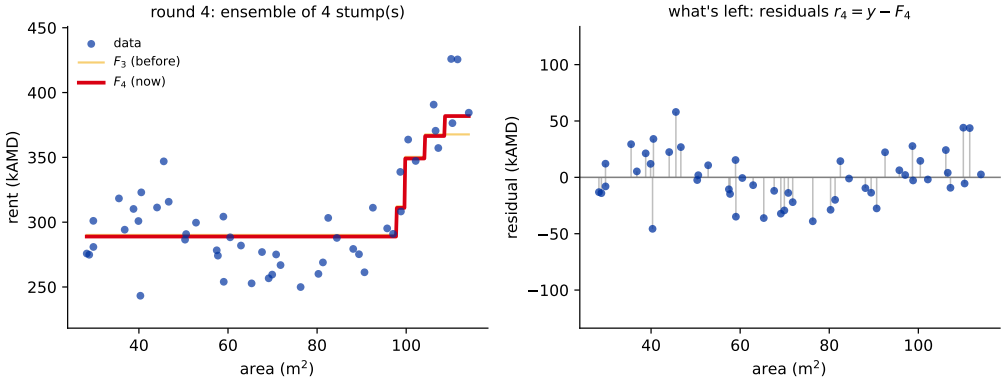
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6.)*

Watch it fit: gradient boosting, round by round



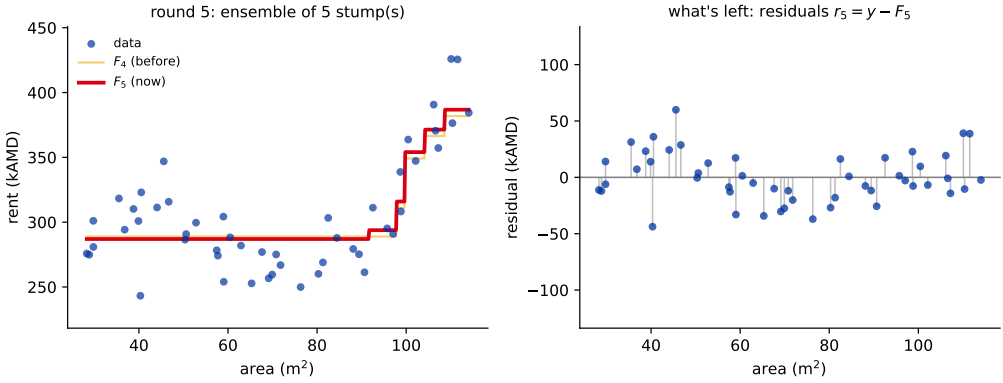
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6.)*

Watch it fit: gradient boosting, round by round



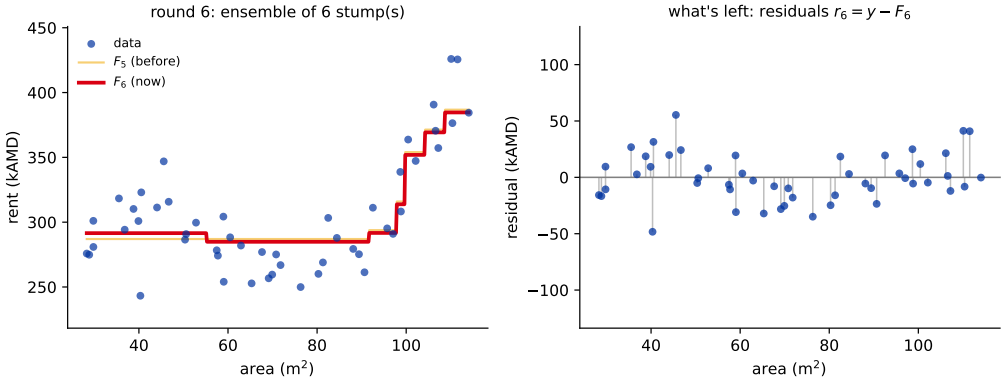
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6.)*

Watch it fit: gradient boosting, round by round



Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6.)*

Watch it fit: gradient boosting, round by round



Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6.)*

Predict-first: does boosting overfit with more trees?

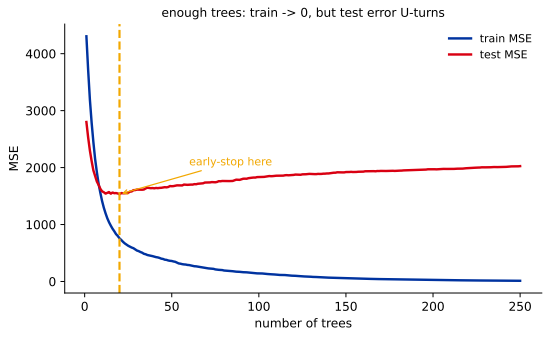
In [18], *more trees never hurt* a random forest. Same for boosting?

Predict-first: does boosting overfit with more trees?

In [18], *more trees never hurt* a random forest. Same for boosting?

No – the opposite. Training error drives toward **0** (it can memorize); test error drops, then **U-turns** and rises.

Boosting keeps fitting the *previous* errors, eventually fitting noise. Controlled by a small η + **early stopping**.



A high-capacity config (depth-3 trees, 250 rounds) on a noisy toy – cranked up to *show* the failure mode.

Contrast with random forests: RF averages *independent* trees → more is safe. Boosting adds *dependent* trees that chase residuals → more can overfit.

The reframe: you were doing gradient descent

Why is fitting residuals a good idea? Look at the squared loss $L = \frac{1}{2}(y - F)^2$:

$$\frac{\partial L}{\partial F} = -(y - F) \quad \Longrightarrow \quad -\frac{\partial L}{\partial F} = y - F = \textbf{the residual}.$$

(The $\frac{1}{2}$ is the convention that makes this come out with no stray factor of 2.)

So the residual you have been fitting **is the negative gradient** of the loss. Each tree takes one step **downhill** – you were doing gradient descent all along, just in *function space* (adding functions) instead of parameter space.

Function-space gradient descent (the general algorithm)

For $m = 1, 2, \ldots, M$:

1. **Pseudo-residuals** (the negative gradient at the current model):

$$r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$$

2. Fit a base learner h_m to the pseudo-residuals r_{im}.
3. **Line search** for the best step: $\gamma_m = \arg\min_\gamma \sum_{i=1}^{n} L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma\, h_m(\mathbf{x}_i))$.
4. Update: $F_m = F_{m-1} + \eta\, \gamma_m\, h_m$.

What sklearn actually does (a refinement): it fits the tree, then sets *each leaf* to its own optimal step (a per-leaf line search), and shrinks all leaves by η. For squared loss that per-leaf step is just the leaf's **residual mean** – exactly the by-hand round you did two slides ago.

Any loss you like

Swap the loss $\Rightarrow$ swap the pseudo-residual. The whole machinery is unchanged; only "what each tree chases" changes.

- **Squared loss** $\rightarrow$ pseudo-residual $= y - F$ (the plain residual).
- **Absolute / Huber loss** $\rightarrow$ pseudo-residual = the *sign* or a *clipped* residual $\rightarrow$ **robust to outliers** (one bad point can't drag the fit far).

This is boosting's superpower over AdaBoost: pick the loss that matches your problem (regression, classification, ranking, robust, quantile) – the algorithm does not change.

Gradient boosting for classification

Same machinery, different loss. For a yes/no target with **log-loss**:

- the model F accumulates in **log-odds** space (not probability);
- the pseudo-residual is simply $y - p$, where $p = \text{sigmoid}(F)$ is the current predicted probability;
- trees are added in log-odds; the **sigmoid is applied once, at the end**, to read off a probability.

Regression chases $y - F$; classification chases $y - p$. Same gradient-descent idea – this is what the Titanic homework runs.

Regularization

More power than you want – three knobs hold it back.

Three knobs

knob	what it controls
`n_estimators` (M)	number of trees / rounds. More = lower bias, but eventually overfits $\Rightarrow$ use **early stopping**.
`max_depth`	per-tree depth = **interaction order** (depth 1 = main effects only; deeper = feature interactions). Boosting likes *shallow*.
`learning_rate` (η)	**shrinkage**: how big a step each tree takes. Smaller = steadier, generalizes better.

The big one is the pair (η, M) – they trade off against each other (next slide).

Predict-first: halve the learning rate – then what?

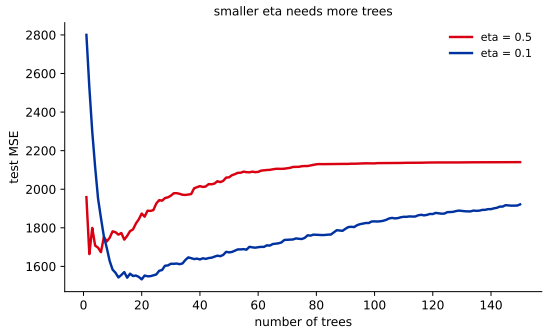
If you **halve** η, what happens to the number of trees you need?

Predict-first: halve the learning rate – then what?

If you **halve** η, what happens to the number of trees you need?

Roughly **double** (a rule of thumb in the small-η regime, not exact).

Small η = slow, smooth, generalizes; large η = fast, jagged, overfits sooner. The standard recipe: **small η + many trees + early stop**.



Smaller η needs more trees – and here reaches a *lower* test error. (General trend; not the exact 2x.)

Stochastic gradient boosting

Add a dash of **bagging**: each round, fit the tree on a random **subsample** of the rows (and optionally columns), not all of them.

- Faster (smaller fits) and a **regularizer** (the randomness decorrelates the trees, callback [18]).
- A bagging + boosting hybrid (Friedman, 2002).

Heads-up: this is NOT on by default – XGBoost's `subsample` and LightGBM's `bagging_fraction` both default to 1.0 (no subsampling). The libraries make it easy; **how they use it is [20]**.

Practice and the bridge ahead

Just enough to run it – the libraries get their own lecture.

Gradient boosting in sklearn (the essentials)

```
from sklearn.ensemble import HistGradientBoostingClassifier

gb = HistGradientBoostingClassifier(
        learning_rate=0.1, max_depth=3,
        early_stopping=True,          # stop when validation stops
            improving
        validation_fraction=0.1, n_iter_no_change=10,
        random_state=509)
gb.fit(X_train, y_train)              # Titanic
```

Early stopping picks M for you (callback: the cross-validation lecture). Recall the single-tree LightGBM from [17] – now it is `n_estimators`=100 with a small η. **Tuning recipes and the XGBoost/LightGBM/CatBoost APIs are [20]** – this just gets gradient boosting running.

Bridge: the deep library session ([20])

Everything so far is the **algorithm**. Next lecture is the **engineering**:

- **XGBoost / LightGBM / CatBoost** – the libraries that win on tabular data: speed (histogram + leaf-wise growth), regularization baked into the objective, native categorical handling.
- How to **tune** them (the knob order that matters) and a real bake-off.
- **Stacking** – combining different model types – to close the chapter.

Next ([20]): same gradient boosting you just learned – made fast, regularized, and production-ready.

Recap

- **Boosting** adds shallow trees **in sequence**, each fitting the previous ensemble's leftover error; it cuts **bias**.
- That "leftover error" is the **negative gradient** of the loss – boosting is gradient descent in function space (any loss: $y - F$ regression, $y - p$ classification).
- More trees **can overfit** (unlike RF) – control with small η, shallow trees, and **early stopping**.

Workflow: small η + many trees + early stopping; keep trees shallow; add row/column subsampling if it helps. For the production versions, see **[20]**.

HW11 – boost the tree

1. **By hand** (Yerevan rent toy): do **3 rounds** of gradient boosting – init $=$ mean(y), compute residuals, fit a stump, update with η. Show the residuals shrinking.
2. **sklearn** (Titanic, chapter project): fit a `GradientBoostingClassifier` / `HistGradientBoostingClassifier` with early stopping; compare to your **L10 random forest** on the same score (accuracy + F1 / ROC-AUC).
3. Tune `learning_rate` (with early stopping picking `n_estimators`).

Bonus: the AdaBoost $\alpha = \frac{1}{2} \ln \frac{1-\text{err}}{\text{err}}$ arithmetic for one round; force overfitting (train $\to 0$, test U) then early-stop; try `subsample` < 1.